



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)

Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltes.com Email: principal@dypatiltes.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil

Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil

Chairman

Adv. Anuja Sushant Patil

Director

Dr. Suresh Shirbahadurkar

Principal

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: C Programming

Prepared By: Sakshi Kunthe

Introduction to C

C was developed at AT&T; Bell Laboratories in the USA in 1972 by Dennis Ritchie. The uploaded book notes its original use in developing UNIX and highlights C's performance, reliability and usefulness for system software and device drivers.

Programming languages can broadly be understood as low-level and high-level languages. The source explains machine language and assembly language as low-level languages, while high-level languages are designed to be more machine independent. It also introduces compiler, interpreter and assembler as translators

Why learn C?

Strong foundation for programming logic, memory and data structures.

Fast execution and close interaction with computer memory.

Widely used in systems, embedded software, operating-system components and performance-sensitive applications.

Provides concepts that make pointers, arrays, structures and dynamic memory easier to understand.

Core idea

A computer program is a collection of instructions used to solve a specific problem. The method used to solve the problem is an algorithm.

2. C Program Structure

The lecture note presents a typical C program as a combination of comments, preprocessor directives, global declarations, the main function, local declarations/statements and user-defined functions.

```
/* First C program */ #include  
<stdio.h>  
  
int main(void) {    printf("Welcome to C  
Programming!\n");    return 0; }
```

Important parts:

- **#include** brings declarations from a header file into the program.

- **main()** is the program entry point in a hosted C program.
- **{ }** delimit a block.
- **;** terminates most C statements.
- **return 0;** reports successful termination from main.

Comments

Comments improve readability and documentation. The source describes block comments using `/* ... */`; modern C also commonly uses `//` for a single-line comment.

3. Compilation and Execution

The source explains a build sequence: write a `.c` source file, compile it, translate it into lower-level/object code, link required libraries and finally execute the resulting program. It describes gcc as a common GNU C compiler command.



Typical GCC commands

```
gcc program.c o program ./program
```

```
# Windows (MinGW  
GCC) gcc  
program.c -o  
program.exe  
program.exe
```

Errors and debugging

Compilation checks syntax and other language constraints. If the program builds but produces the wrong result, inspect the algorithm and logic during debugging. The source specifically describes debugging as returning to the source program to remove known problems.

IDE

An Integrated Development Environment combines editing, compiling, linking, running and debugging in one application.

4. Character Set, Identifiers and Keywords

A character in C may be an alphabet, digit or special symbol. When these are combined according to C's rules, they form constants, variables, keywords and program statements.

Identifiers

- Used to name variables, arrays, functions, structures and other program entities.
- May contain letters, digits and underscore.
- The first character must be a letter or underscore.
- Cannot be a C keyword.
- C is case-sensitive: value, Value and VALUE are different identifiers.
- Prefer meaningful names such as total, marks, studentName and average.

Examples

```
int age; float  
average_m
```

```
arks; char  
grade;
```

```
int 2value;    // invalid: starts with a  
digit int total-mark; // invalid: '-' is  
not allowed int int;      // invalid:  
keyword
```

Keywords

Keywords are reserved words with predefined meaning, such as int, char, float, double, if, else, for, while, return, struct, union, static, extern and const.

5. Data Types, Variables and Constants

The source classifies C data types into basic built-in types (int, float, double, char), enumeration, derived types (pointer, array, structure, union) and void. It also discusses short/long and signed/unsigned qualifiers.

Type	Typical use	Example
int	whole numbers	int count = 10;
float	single-precision real values	float price = 12.5f;
double	higher precision real values	double pi = 3.14159;
char	one character	char grade = 'A';
void	no value / no type value	void display(void);

Variables

A variable is a named storage location whose value can change during execution. It should be declared before use.

```
int age = 20; float
salary =
25000.50f; char
grade =
'A'; age = 21;    // value can change
```

Constants

A constant represents a value intended not to change during program execution. The source covers numeric, character, string and symbolic constants.

6. Constants, Strings and Escape Sequences

Numeric constants may be integers or floating-point values. The source discusses decimal, octal and hexadecimal integer forms and scientific notation for real constants.

```
int a = 25;    // decimal int
b = 012;    // octal int c =
0x2A;    // hexadecimal
double d = 3.6e5; //
scientific notation
char ch = 'A'; char
*msg =
"Hello";
```

Character vs string

A character constant uses single quotes, e.g. 'A'. A string literal uses double quotes, e.g. "A", and is stored with a terminating null character (\0). The source explicitly distinguishes these two forms.

Symbolic constants

```
#define MAX
```

```
100 #define
```

```
PI
```

```
3.14159265358 9793
```

```
const int DAYS =
```

```
7;
```

Useful escape sequences

Sequence	Meaning
\n	new line
\t	horizontal tab
\\	backslash
\"	double quote
\0	null character

7. Input and Output

The source identifies `printf()` for output and `scanf()` for input and introduces `stdio.h` as the standard input/output header.

Common format specifiers

Specifier	Typical data	Example
%d	int	printf("%d", n);
%f	floating-point output	printf("%f", x);
%lf	double with scanf	scanf("%lf", &x);
%c	char	printf("%c", ch);
%s	string	printf("%s", name);
%u	unsigned int	printf("%u", n);

Input example

```
#include
<stdio.h>

int main(void) {    int
age;    printf("Enter
age: ");
scanf("%d",
&age);
printf("Age = %d\n",
age);    return 0;
```

Remember: scanf normally receives the address of a variable, hence &age; for an int variable.

8. Expressions and Operators

An expression combines variables, constants, operators and/or function calls. The source gives arithmetic, relational and functioncall examples.

Operator categories

Category	Operators	Purpose
Arithmetic	+ - * / %	numeric calculations
Assignment	= += -= *= /= %=	store/update values
Relational	< <= > >= == !=	compare values
Logical	&& !	combine conditions
Increment/decrement	++ --	change by one
Conditional	?:	short two-way choice
Bitwise	& ^ ~ << >>	bit-level operations
Size	sizeof	size in bytes
Comma	,	evaluate expressions left-to-right

9. Assignment, Relational and Logical Operators

Assignment stores the right-hand value/expression in the left-hand variable. Compound assignment is a shorter form of update. `int x`

```
= 10; x += 5; // x
```

```
= 15 x *=
```

```
2; // x =
```

```
30 x -= 4;
```

```
// x = 26
```

Relational operators

Relational expressions evaluate to a true/false result (in C, zero represents false and nonzero represents true). $a > b$ a

$< b$ $a == b$ a

$!= b$

Logical operators

$(age >= 18) \&\& (citizen == 1)$

$(mark >= 40) || (attendance >= 75)$

$!(x == 0)$

Conditional operator

`int max =`

`(a`

`> b) ? a :`

`b;`

Comma operator

The comma operator evaluates expressions from left to right and is often seen in for-loop initialization/update expressions. The source discusses this use.

10. Increment, Decrement and sizeof

`++` increases a variable by one and `--` decreases it by one. They can be prefix or postfix. The source explains that prefix changes the value before it is used, while postfix uses the old value first.

```
int x = 5; int a =  
++x; // x=6, a=6
```

```
int y = 5; int b = y++; //  
b=5,  
y=6
```

sizeof `sizeof` is a unary operator that gives the size of a type or object in bytes. The source highlights its usefulness for portable programs and dynamic allocation. `printf("%zu\n", sizeof(int)); printf("%zu\n", sizeof(double));`

```
int a[10];  
printf("%zu\n",  
sizeof(a));
```

Do not confuse `=` with `==`

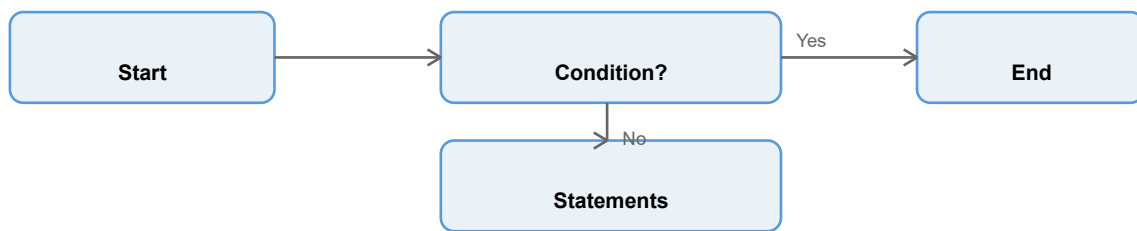
```
if (x == 10) //  
comparison  
printf("Ten"); x = 10;  
// assignment
```

11. Operator Precedence and Associativity

When an expression contains multiple operators, precedence determines which operator binds more strongly. Associativity resolves operators of the same precedence. The source provides a precedence table from function calls/array subscripts through

arithmetic, shifts, comparisons, logical operators, conditional, assignment and comma.

High → Low	Operators
1	() [] . ->
2	+ - ++ -- ! ~ * & sizeof (type)
3	* / %
4	+ -
5	<< >>
6	< <= > >=
7	== !=
8–10	& ^
11–12	&&
13	? :
14	= += -= *= /= %= ...
15	,



Best practice: use parentheses when an expression is difficult to read. Clear code is safer than relying on memorized precedence.

12. Control Statements

Normally statements execute in sequence. Control statements change the flow using decisions, loops and jumps. The source lists if/else, switch, while, do-while, for, break, continue and goto.

Main categories

- **Selection:** if, if-else, else-if ladder, switch.
- **Iteration:** while, do-while, for.
- **Jump:** break, continue, goto, return.

General decision pattern

```

if (condition) {
// true block
} else { // false
block
}
  
```

13. if, if-else and else-if Ladder

The if statement executes a statement/block only when its condition is true. An if-else provides two possible paths. The source explains that zero is treated as false and a nonzero value as true.

```

if (marks >=
40) {
printf("Pass"); }
  
```

```
else {  
printf("Fa il"); }
```

Else-if ladder

```
if (marks >= 90) grade = 'A'; else if
```

```
(marks >= 75) grade = 'B'; else if
```

```
(marks >= 60) grade = 'C'; else  
grade = 'D';
```

Nested if

When one if-else appears inside another if/else block, it is called nesting. It is useful when a second condition should be tested only after a first condition is satisfied.

Example: largest of two numbers

```
if (a > b) printf("%d  
is larger", a); else  
printf("%d is  
larger", b);
```

14. switch Statement

The source syllabus includes decision making and branching, and its control-statement section lists switch. switch is useful when one expression must be matched against several constant case values.

```
switch (choice) { case 1:  
printf("Add ");  
break; case  
2: printf("Vie  
w"); break;  
case
```

```

3: printf("Exit");
break;    default:
printf("Invalid
choice"); }

```

Key points

- case labels identify possible constant values.
- break prevents execution from falling through into the next case.
- default handles unmatched values.
- Use switch for menu-style or discrete choices; use if/else for ranges and complex conditions.

Menu skeleton

```

int choice;
printf("1.Add 2.View
3.Exit\n");
scanf("%d",
&choice);

switch(choice) {    case 1: /*
... */ break;    case 2: /* ... */
break;    case 3: /* ... */
break;    default:
printf("Invalid choice");
}

```

15. while and do-while Loops

A loop repeatedly executes a block while a condition remains satisfied. The source identifies while, do-while and for as the three main loop forms. **while** int i = 1; while (i <= 5) { printf("%d ", i); i++;

} **do-while**

```
int choice; do {    printf("Enter 1 to  
continue, 0 to stop: ");    scanf("%d",  
&choice); } while  
(choice == 1);
```

Difference

- while checks the condition before the first iteration.
- do-while executes the body first, so it executes at least once.
- The do-while statement ends with a semicolon.

The source explicitly emphasizes this at-least-once behavior of dowhile.

16. for Loop, break and continue for is generally used when the number of iterations is known or can be expressed by a counter. Its three expressions are initialization, condition and update for (int i = 1; i <= 10; i++) { printf("%d ", i); }

Nested loops

A loop inside another loop is a nested loop. The source shows nested for loops where the outer loop handles one index and the inner loop handles another.

break

Immediately terminates the nearest loop or switch and transfers control after it

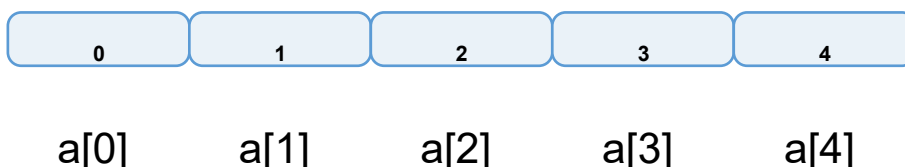
continue

Skips the remaining body of the current iteration and proceeds to the next iteration. In a for loop, control reaches the update expression before the next condition test. for (int i=1; i<=10; i++)
{ if (i == 5) continue; if (i == 9) break; printf("%d ", i);
}

17. One-Dimensional Arrays

An array is a collection of elements of the same data type stored in contiguous memory. The first index is 0. A one-dimensional array is often called a vector.

1-D Array: contiguous elements



Declaration and initialization

```
int marks[5]; int values[5] = {10, 20,  
30, 40, 50}; int a[] =  
{2, 4, 6, 8};
```

Access

```
printf("%d", values[0]); // first
element values[2] = 99; //
change third element
```

Important rule

For an array declared as `int a[5]`, valid indices are 0 through 4. Accessing outside the valid range causes undefined behavior.

18. Array Processing and Memory

The source demonstrates reading array elements with a for loop and displaying them with another loop. It also explains that array elements occupy consecutive memory locations.

```
int arr[5], i;
```

```
for (i = 0; i < 5; i++) {
    printf("Enter arr[%d]: ",
    i);    scanf("%d",
    &arr[i]); }
```

```
for (i = 0; i < 5; i++)
    printf("%d ", arr[i]);
```

Common array operations

- Traversal: visit every element.
- Searching: find whether/where a value occurs.
- Aggregation: sum, average, minimum and maximum.
- Sorting: arrange elements in ascending/descending order.
- Copying: copy elements one-by-one or with appropriate library/memory functions.

Sum example

```
int sum = 0;
```

```
for (= 0; i &lt; 5;
i++)
    sum += arr[i];
printf("Sum=%d",
sum);
```

19. Two-Dimensional Arrays

A two-dimensional array is commonly used to represent a matrix. It uses row and column subscripts. The source gives the form data-type array[row][column] and notes that the number of elements is row × column.

```
int mat[2][3] =
{ {20, 2,
7},
{8, 3, 15}
};
```

Accessing with nested loops

```
for (int i = 0; i &lt; 2; i++) {
for (int j = 0; j &lt; 3; j++) {
printf("%d ", mat[i][j]);    }
printf("\n"); }
```

Matrix processing

- Row-wise and column-wise traversal.
- Matrix addition/subtraction.
- Transpose.
- Finding diagonal elements.
- Matrix multiplication (when dimensions are compatible).

The source emphasizes an outer loop for rows and an inner loop for columns.

20. Strings in C

A string is a sequence of characters terminated by the null character `\0`. The source treats an array of characters as a string and explains that double-quoted string literals include the terminating null character.

```
char name[20]; scanf("%19s", name);
```

```
printf("Name = %s",  
name);
```

Safer line input

```
char line[100];  
fgets(line, sizeof line,  
stdin); printf("%s",  
line);
```

String header

String library functions are covered in the uploaded lecture note as part of Module II.

Function	Purpose
<code>strlen(s)</code>	length of string (excluding <code>\0</code>)
<code>strcpy(d,s)</code>	copy string <code>s</code> to <code>d</code>
<code>strcat(d,s)</code>	append <code>s</code> to <code>d</code>
<code>strcmp(a,b)</code>	compare two strings
<code>strchr(s,c)</code>	find a character
<code>strstr(s,t)</code>	find a substring

21. String Library Examples

The source specifically explains `strcpy()` and `strcat()` and includes string handling in its module plan.

```
<stdio.h>
```

```
#include
<string.h>
;

int main(void) {    char first[50] = "Hello";    char second[] =
" World";    printf("Length = %zu\n", strlen(first));

    strcat(first, second);
    printf("%s\n", first);

    return 0;
}
```

Important memory rule

The destination array passed to strcpy/strcat must have enough space for the resulting characters and the terminating \0. C does not automatically prevent buffer overflow.

Comparing strings

```
if
(strcmp(a,
b) == 0)    printf("Sam
e"); else
printf("Differe n t");
```

Exam tip

Do not compare C strings with ==. Use strcmp() when you want lexical/content comparison.

22. Functions: Basics and Prototypes

A function is a self-contained block of code that performs a specific task. The source covers user-defined functions, function declaration/prototype, calls, arguments, categories and modular programming.

Prototype

```
int add(int a, int  
b);
```

Definition and call

```
int add(int a,  
int b) {  
    return a + b; }  
  
int main(void) {    int  
    result = add(10, 20);  
    printf("%d", result);  
    return 0; }
```

Why functions?

- Break a large problem into smaller tasks.
- Improve readability and testing.
- Enable reuse of logic.
- Make collaborative development easier.

The source contrasts monolithic programming with modular programming and lists maintainability, testing and reuse as key concerns.

23. Function Categories and Arguments

The uploaded notes present four common categories based on arguments and return value: no argument/no return, argument/no return, no argument/return, and argument/return.

Category	Prototype example	Call
No argument, no return	<code>void show(void);</code>	<code>show();</code>
Argument, no return	<code>void show(int x);</code>	<code>show(10);</code>
No argument, return	<code>int read(void);</code>	<code>x=read();</code>
Argument, return	<code>int add(int,int);</code>	<code>x=add(a,b);</code>

Formal vs actual arguments

Formal parameters are the parameters listed in the function definition. Actual arguments are the values/expressions supplied in the function call. The source explicitly describes formal arguments as local to the called function.

24. Call by Value, Reference-style Passing and Recursion

In C, ordinary parameters are passed by value: the called function receives copies. To modify the caller's variable, pass its address using a pointer. The source describes this as call by reference and demonstrates pointer parameters.

```
void swap(int *a, int *b) {    int temp =
*a;    *a =
*b;
    *b
= temp;
```

```

}
int main(void) {
int x=10, y=20;
swap(&x,
&y); }

```

Recursion

When a function calls itself, it is recursive. The source demonstrates factorial recursion.

```

int factorial( int n) {    if (n &lt;= 1)        return 1;
return n * factorial(n - 1); }

```

Every recursive solution needs a base case; otherwise calls continue indefinitely until the program exhausts its call stack.

25. Local, Global, Static and Storage Classes

The source discusses local, global and static variables and then lists automatic, register, static and extern storage classes.

Storage class	Scope	Typical lifetime	Key idea
auto	block/local	until block ends	default local variable
register	block/local	until block ends	request fast/register storage
static (local)	block	entire program	retains value between calls
extern	global across files	entire program	refers to external definition

Static example

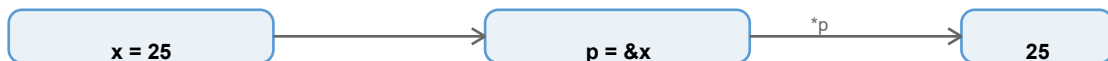
```
void counter(void)
{
    static int x = 0;
    x++;    printf("%d\n", x); }

/* calls print
1, 2, 3 */
```

The source notes that static variables are initialized once and retain their value between function calls.

26. Pointers: Foundation

A pointer variable stores the address of another object. The source introduces the address operator `&` and indirection/dereference operator `*`.



```
int x = 25; int *p
= &x;

printf("%d\n", x);
printf("%d\n", *p);

*p = 40;
// changes x to
40
```

Declaration syntax

data_type

```
*pointer_name; int *p;  
char *ch; double *d;
```

Why pointers matter

- Pass addresses to functions to modify caller variables.
- Traverse arrays efficiently.
- Build dynamic data structures.
- Allocate and release dynamic memory.
- Access structure members through pointer-to-structure syntax.

27. Pointer Arithmetic and Comparisons

Pointer arithmetic depends on the pointed-to type. For an int pointer, `p+1` advances by `sizeof(int)` bytes. The source covers increment/decrement, subtraction of pointers within the same array and pointer comparisons. `int a[] = {10, 20, 30, 40}; int`

```
*p = a;
```

```
printf("%d\n", *p);    //  
10 printf("%d\n", *(p +  
2)); // 30
```

```
p++; printf("%d\n", *p);  
// 20
```

Valid pointer arithmetic

- Add/subtract an integer from a pointer.
- Increment/decrement a pointer.
- Subtract two pointers that point into the same array.
- Compare pointers when the comparison is meaningful (commonly within the same array).

Not valid

Adding two pointers or multiplying/dividing pointers is not meaningful C pointer arithmetic.

28. Pointer to Pointer and Arrays

A pointer to pointer stores the address of another pointer. The source gives the syntax `data_type **p` and an example where `int **p1` points to an int pointer.

```
int x = 22; int
*p = &x;
int **pp
= &p;

printf("%d\n", **pp); //
22
```

Array and pointer relationship

In many expressions, an array name converts to a pointer to its first element. This is why `arr[i]` and `*(arr+i)` access the same element for a normal array. `int arr[] = {10,20,30};`

```
printf("%d\n", arr[1]);
printf("%d\n", *(arr +
1));
```

Pointer-to structure

```
struct Student {
int roll;      float
marks; };

struct Student s = {1,
88.5f}; struct Student
```

```
*p = &s;  
printf("%d %.1f", p-  
>roll, p-  
>marks);
```

29. Structures

A structure groups members of different data types into one record. The source calls it a collection of dissimilar/heterogeneous data types and gives student records as an example.

```
name[30] roll age marks[5]
different data types grouped as one record
```

struct student

```
struct
Student {
char
name[30] ;
int roll; int
age; float
marks;
}; struct Student s1 = {"Teju", 101, 20,
88.5f};
```

Accessing members

```
printf("%s", s1.name);
printf("%d", s1.roll);
s1.age
= 21;
```

Array of structures

```
struct Student
students[100];

students[0].roll
= 101; students[1].roll
= 102;
```

The source also covers arrays inside structures and nested structures.

30. Passing Structures to Functions and Nested Structures The lecture note demonstrates passing an entire structure to a function and explains that individual members can also be passed.

```
struct Student {    char name[30];    int age;    int roll; };
```

```
void display(struct Student s) {  
    printf("%s %d %d\n",  
        s.name, s.age, s.roll);  
}
```

```
int main(void) {    struct Student s =  
    {"Riya", 20, 101};    display(s); }
```

Nested structure

```
struct Date {    int  
    day, month, year;  
};
```

```
struct Student {  
    char name[30];  
    int roll;    struct  
    Date dob;  
}; struct Student s =
```

```
{"Riya",  
101, {12, 11, 2005}};
```

Pointer to structure

Use -> when a pointer is used to access a structure member: ptr->roll.

Use . when you have the structure object itself: s.roll.

31. Unions

A union resembles a structure in syntax, but its members share the same memory area. The source highlights its use when only one member is needed at a time and notes its memory-saving property.

```
union Data {    int i;    float f;    char text[20] ; };
```

```
union Data d; d.i  
= 10; printf("%d  
", d.i);
```

Structure vs union

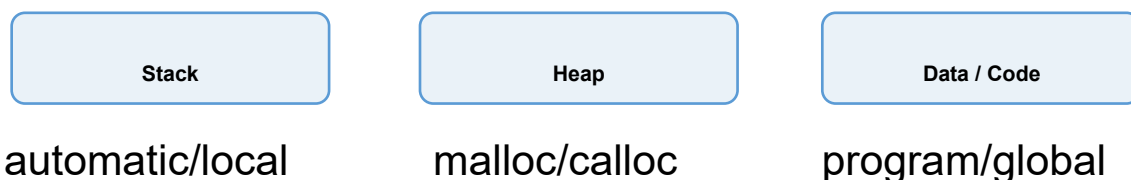
Feature	Structure	Union
Memory	members have separate storage	members share storage
Simultaneous values	all members can hold values	normally one active value at a time
Use	records with multiple fields	memory-efficient alternative representations

Nested unions

The source also discusses nesting a union inside a structure or a structure inside a union.

32. Dynamic Memory Allocation

Dynamic memory allocation obtains memory at run time. The source covers malloc(), calloc(), realloc() and free(), with malloc returning a pointer to allocated memory and free releasing it.



malloc

```
int n = 5; int *p =  
malloc(n * sizeof *p);  
  
if (p == NULL) {  
    printf("Allocation  
failed");    return 1;  
}  
for (int i=0; i<n;  
i++)    p[i] = i + 1;  
free(p) ;
```

calloc

```
int *p = calloc(5, sizeof  
*p);
```

Difference

- malloc(n) allocates n bytes; allocated bytes are not initialized.
- calloc(count, size) allocates count × size bytes and initializes the allocated bytes to zero.
- free(pointer) releases dynamically allocated memory.

Always check allocation results and release memory when it is no longer needed.

33. File Handling

A file provides persistent storage. The source covers sequential, random-access and binary files and explains that C communicates with files using a FILE pointer.



Opening a file

FILE

```
*fp;
```

```
fp =
```

```
fopen("data.txt"  
 , "r");
```

```
if (fp == NULL)
```

```
{    printf("Unable to  
open file");    return 1;  
}
```

Mode	Meaning
r	read existing file
w	write; creates/truncates file
a	append to file
rb	read binary
wb	write binary

Always close a successfully opened file with `fclose(fp)`. The source explicitly introduces `fopen()`, NULL failure checking and closing. File Read/Write Functions

The source covers character and string file operations including `getc()`, `putc()`, `fgets()` and `fputs()`, and gives a file-copy example.

Character-by-character copy

```
FILE          *src          =
```

```
fopen("source.txt",  
"r"); FILE *dst =
```

```
fopen("dest.txt",  
"w");
```

```

if (src == NULL || dst
== NULL)
{   printf("File error");
return 1; }

```

```

int ch; while ((ch =
fgetc(src)) != EOF) {
fputc(ch, dst);
}

```

```

fclose(src)
;
fclose(dst)
;

```

String operations char line[200];

```

while (fgets(line, sizeof line, src)
!=
NULL) {   fputs(line, dst); }

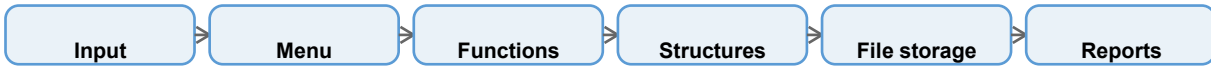
```

Useful functions

Function	Purpose
fopen()	open a file
fclose()	close a file
fgetc()	read one character
fputc()	write one character
fgets()	read a line/string
fputs()	write a string
fprintf()	formatted output to file
fscanf()	formatted input from file

34. C Mini Project — Student Record Management System

This project combines the major topics in the uploaded syllabus: input/output, control statements, arrays, strings, functions, structures, pointers and file handling. The syllabus explicitly lists these areas across Modules I–III.



Features

- Add a student record.
- Display all records.
- Search by roll number.
- Update marks/details.
- Store records in a binary file.
- Load/display records whenever the program starts.

Data model

```
struct
Student {
int      roll;
char
name[40] ;
float marks;
};
```

Suggested menu

1. Add student
2. Display students
3. Search student
4. Exit

Project flow main() → display menu → read choice → call a dedicated function → perform structure/file operation → return to menu.

This is a modular design rather than one large monolithic function.

Starter implementation idea

```
void addStudent(void); void
displayStudents(void); void
searchStudent(void);

int main(void)
{
    int choice; do { printf("\n1.Add 2.Display
3.Search 4.Exit\n"); scanf("%d",
&choice);

        switch(choice) {
case 1: addStudent(); break;
case 2:
displayStudents(); break;
case 3:
searchStudent();
break; case 4:
break; default:
printf("Invalid choice");
}
}
```

```
    } while(choice !=  
4);  
  
    return  
0; }
```

Project extension ideas: validation, sorting by marks, CSV export, login/admin role, multiple files, and a report showing average/highest/lowest marks.

35. Exam Quick Revision Sheet

Core syntax to memorize

// Variable

```
int x  
=  
10;
```

// if if

```
(condition) { ... } else {  
...  
}
```

```
// switch switch (choice) { case 1:  
... break; default: ... }
```

// loops while

```
(condition) { ...  
} do { ... } while (condition); for  
(init; condition; update) { ... }
```

```
// array int a[5]  
= {1,2,3,4,5}  
;
```

```
// string char  
name[30]  
=  
"Teju";
```

```
// function return_type name(type p1, type p2) { ... } //
pointer int *p = &x;

// structure struct Student { int roll;
char name[30]; };

// dynamic memory int
*p = malloc(n * sizeof
*p); free(p);

// file FILE *fp
= fopen("data.txt" , "r");
fclose(fp);
```

Common mistakes

- Using = when == is required.
- Forgetting & in scanf for scalar variables.
- Going outside array bounds.
- Forgetting the string's terminating \0 or allocating too little space.
- Dereferencing an uninitialized or NULL pointer.
- Forgetting free() after dynamic allocation.
- Forgetting fclose() after file operations.
- Forgetting break in switch when fall-through is not intended.
- Writing a recursive function without a base case.

36. Topic Map and Final Checklist

The uploaded book's contents are organized into three modules: Module I covers C fundamentals, operators, control statements and arrays; Module II covers strings, functions, arguments, recursion, storage classes and pointers; Module III covers structures, unions, dynamic memory allocation and file handling.

Module	Must-know topics
I	C introduction, program structure, compilation, character identifiers, keywords, data types, constants, variables, expressions, operator
II	String functions, function declaration/call/definition, four categories, actual/formal arguments, value/address-based parameter pas
III	Structures, arrays of structures, nested structures, pass structures to functions, unions, nested unions, dynamic allocation, dyn

Final checklist

- . Can I write and compile a basic C program?
- . Can I use printf/scanf and format specifiers correctly?
- . Can I solve a problem using if, switch and loops?
- . Can I process 1-D/2-D arrays and strings?
- . Can I write, call and prototype functions?
- . Can I explain call-by-value and address-based modification?
- . Can I trace recursion?
- . Can I explain pointer, * and &?
- . Can I define/use structures and unions?
- . Can I allocate/free dynamic memory?
- . Can I open, read, write and close files?
- . Can I combine these into a modular C project?

Source note: These notes are based primarily on the uploaded 127page MCA 101 Programming in C lecture note. Some examples in the source contain typographical/legacy C issues; this study guide uses standard modern C syntax in the examples while preserving the source's topic coverage and terminology. The uploaded document itself is marked as under revision.